

Big Data Analytics & Visualization

LAB 6: PIG & HIVE

Practical Data Analysis Using Apache Pig on Hadoop

Prepared by:

BOUHOUIA Yousra G1

Academic Year 2024/2025

1 Introduction

In this lab, we worked on employee data in a Hadoop environment and performed analytical processing using Apache Pig. The practical work focused on preparing the datasets in HDFS, then using Pig to analyze employees, contributions, and reimbursements.

We first created a personal directory in HDFS and uploaded the three required CSV files. After verifying that the files were stored correctly, we moved to Part 2 and solved the required questions using Pig.

The three datasets used in this lab are:

- employees.csv
- employee_contributions_small.csv
- employee_reimbursements.csv

2 Environment Preparation

At the beginning of the lab, we prepared our workspace in HDFS by creating a personal directory and uploading the three CSV files into it. This step was necessary to make the files available for Pig processing.

The commands used during this step are the following:

```
hdfs dfs -mkdir -p /user/student207/shared
hdfs dfs -put ~/employees.csv /user/student207/shared
hdfs dfs -put ~/employee_contributions_small.csv /user/student207/shared
hdfs dfs -put ~/employee_reimbursements.csv /user/student207/shared
hdfs dfs -ls /user/student207/shared
```

These commands allowed us to create a shared folder in HDFS and store all the required datasets in it. The verification step confirmed that the files were available and ready to be used in the second part of the lab.

```
student207@hadoop-master:~$ hdfs dfs -mkdir -p /user/student207/shared
hdfs dfs -put ~/employees.csv /user/student207/shared
hdfs dfs -put ~/employee_contributions_small.csv /user/student207/shared
hdfs dfs -put ~/employee_reimbursements.csv /user/student207/shared
student207@hadoop-master:~$ hdfs dfs -ls /user/student207/shared
Found 3 items
-rw-r--r--  3 student207 hadoop_students      8249 2026-04-21 14:51 /user/student207/shared/employee_contributions_sma
ll.csv
-rw-r--r--  3 student207 hadoop_students      1663 2026-04-21 14:51 /user/student207/shared/employee_reimbursements.cs
v
-rw-r--r--  3 student207 hadoop_students      1268 2026-04-21 14:51 /user/student207/shared/employees.csv
student207@hadoop-master:~$
```

Figure 1: Verification of the uploaded files in HDFS

3 Part 2: Practical Questions Using Pig

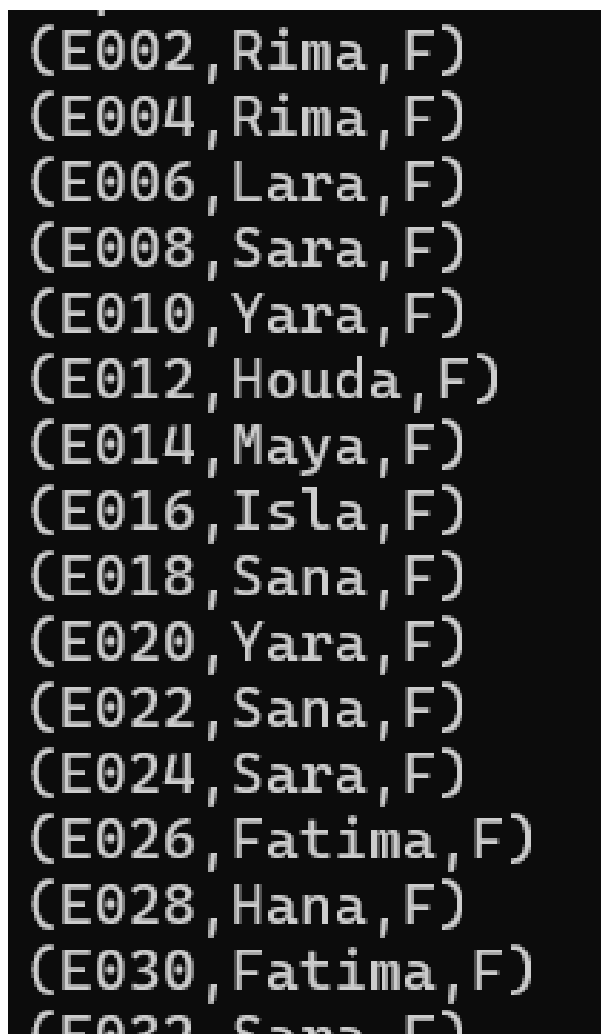
In this part, we analyzed the employee, contributions, and reimbursements datasets using Apache Pig. The goal was to answer a set of practical questions related to filtering, counting, joining, and aggregation.

3.1 Question 1: Show all female employees

This task consists of displaying only female employees from the employees dataset.

To do this, we work on the employees table and focus on the `gender` field. The idea is to keep only the rows where the gender corresponds to female employees. This is a simple filtering operation and it helps us extract only the records that satisfy the requested condition.

The result obtained for this question is shown below.



```
(E002,Rima,F)
(E004,Rima,F)
(E006,Lara,F)
(E008,Sara,F)
(E010,Yara,F)
(E012,Houda,F)
(E014,Maya,F)
(E016,Isla,F)
(E018,Sana,F)
(E020,Yara,F)
(E022,Sana,F)
(E024,Sara,F)
(E026,Fatima,F)
(E028,Hana,F)
(E030,Fatima,F)
(E032,Sara,F)
```

Figure 2: Result of Question 1: Show all female employees

Execution analysis of Query 1 in YARN and MapReduce

For this query, Pig first analyzed the script and detected that the operation used was a FILTER. This means that the system understood that the query only needed to read the employees file and keep only the rows corresponding to female employees.

The following log line shows this clearly:

```
2026-04-21 14:59:02,270 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig
features used in the script: FILTER
```

After that, Pig translated the query into a MapReduce execution plan. The next lines show that Pig created only one MapReduce plan for this query, both before and after optimization:

```
2026-04-21 14:59:02,330 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer -
MR plan size before optimization: 1
```

```
2026-04-21 14:59:02,330 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer -
MR plan size after optimization: 1
```

This means that Query 1 was simple enough to be executed as a single MapReduce job.

Then Pig contacted YARN through the ResourceManager in order to request cluster resources and submit the job for execution. This is shown by the following line:

```
2026-04-21 14:59:02,372 [main] INFO
org.apache.hadoop.yarn.client.DefaultNoHARMFailoverProxyProvider - Connecting to
ResourceManager at /192.168.20.153:8032
```

This step is important because YARN is responsible for allocating resources, launching the job, and monitoring its execution on the cluster.

After connecting to YARN, Pig prepared the execution environment and announced that one MapReduce job was ready for submission:

```
2026-04-21 14:59:02,779 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 1
map-reduce job(s) waiting for submission.
```

Next, Hadoop prepared the input file for processing. The log shows that only one file was processed and that the job was divided into one split:

```
2026-04-21 14:59:02,987 [JobControl] INFO
org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input files to
process : 1
```

```
2026-04-21 14:59:03,114 [JobControl] INFO org.apache.hadoop.mapreduce.JobSubmitter
- number of splits:1
```

This means that the employees dataset was processed as one input part, so the job used one mapper.

After that, the application was submitted to YARN and a tracking URL was created for monitoring the job:

```
2026-04-21 14:59:03,475 [JobControl] INFO
org.apache.hadoop.yarn.client.api.impl.YarnClientImpl - Submitted application
application_1770582555454_1637
```

```
2026-04-21 14:59:03,497 [JobControl] INFO org.apache.hadoop.mapreduce.Job - The url
to track the job: http://hadoop-master:8088/proxy/application_1770582555454_1637/
```

Pig then reported which aliases were involved in the execution:

```
2026-04-21 14:59:03,497 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -
HadoopJobId: job_1770582555454_1637
```

```
2026-04-21 14:59:03,497 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -
Processing aliases emp,q1
```

This confirms that the job was executing the employees relation together with the filtered relation q1.

The progress of the job was then updated by YARN as follows:

```
2026-04-21 14:59:03,510 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -
0% complete
```

```
2026-04-21 14:59:12,636 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -  
50% complete
```

```
2026-04-21 14:59:13,956 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -  
100% complete
```

These lines show the different execution stages of the job until completion.

At the end, Pig reported the final result and job statistics:

```
2026-04-21 14:59:13,959 [main] INFO
```

```
org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:
```

```
Success!
```

```
job_1770582555454_1637 1 0 ... emp,q1 MAP_ONLY ...
```

```
Successfully read 100 records (1658 bytes) from:
```

```
"/user/student207/shared/employees.csv"
```

```
Successfully stored 50 records (1128 bytes) in:
```

```
"hdfs://192.168.20.153:9000/tmp/temp-2050142169/tmp-1956659894"
```

```
Total records written : 50
```

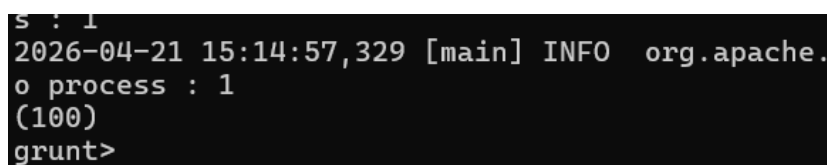
These final lines are very important because they show that:

- the query was executed successfully,
- the job used **1 mapper** and **0 reducers**,
- it was therefore a **MAP_ONLY** job,
- the system read **100 employee records**,
- and produced **50 female employee records**.

So, from a YARN and MapReduce point of view, Query 1 was executed as a simple filtering job. Pig translated the filter into one MapReduce job, YARN allocated the required resources, Hadoop processed the employees dataset, and the final output contained only the female employees.

3.2 Question 2: Count all employees

In this question, we determine the total number of employees present in the employees dataset. This gives us the overall size of the employee table before moving to more advanced operations. We found that the dataset contains 100 employees.



```
s : 1
2026-04-21 15:14:57,329 [main] INFO org.apache.
o process : 1
(100)
grunt>
```

Figure 3: Result of Question 2: Count all employees

3.3 Question 3: Show employees with their total contributions

This task aims to display each employee together with the total amount of contributions associated with that employee.

To achieve this, we first use the contributions table and group the data by employee identifier. After grouping the records, we calculate the sum of the contribution amounts for each employee. Once the total contributions are computed, we combine this result with the employees table so that we can display both the employee information and the total contribution value in the same output.

This step is important because it connects two different datasets. The employees table provides the identity of each employee, while the contributions table contains the financial values. By merging them, we obtain a clearer analytical view of the contribution made by every employee.

```
o process 1
(E001,Walid,3495.36)
(E002,Rima,2353.08)
(E003,Ryad,1962.79)
(E004,Rima,3181.63)
(E005,Karim,5035.84)
(E006,Lara,3558.29)
(E007,Ryad,4054.86)
(E008,Sara,6709.469999999999)
(E009,Hichem,6354.280000000001)
(E010,Yara,1653.61)
(E011,Fares,5296.01)
(E012,Houda,2765.84)
(E013,Anis,2566.02)
(E014,Maya,6485.7)
(E015,Henry,2135.35)
(E016,Isla,2033.23)
(E017,Ryad,6807.58)
```

Figure 4: Result of Question 3: Employees with their total contributions

3.4 Question 4: Show employees with contributions greater than 1000

In this question, we identify the employees whose total contributions are greater than 1000. This allows us to focus only on employees with higher contribution values.


```
o process : 1
(E001,Walid,3495.36)
(E002,Rima,2353.08)
(E003,Ryad,1962.79)
(E004,Rima,3181.63)
(E005,Karim,5035.84)
(E006,Lara,3558.29)
(E007,Ryad,4054.86)
(E008,Sara,6709.469999999999)
(E009,Hichem,6354.280000000001)
(E010,Yara,1653.61)
(E011,Fares,5296.01)
(E012,Houda,2765.84)
(E013,Anis,2566.02)
(E014,Maya,6485.7)
(E015,Henry,2135.35)
```

Figure 5: Result of Question 4: Employees with contributions greater than 1000

3.5 Question 5: Show employees with total reimbursements equal to 0

This task is about finding employees who did not receive any reimbursement. In other words, it highlights employees for whom no reimbursement data exists in the reimbursements dataset.

```
2026-04-21 15:
o process : 1
(E002,Rima)
(E005,Karim)
(E007,Ryad)
(E009,Hichem)
(E011,Fares)
(E013,Anis)
(E014,Maya)
(E015,Henry)
(E017,Ryad)
(E018,Sana)
(E019,Bob)
(E022,Sana)
```

Figure 6: Result of Question 5: Employees with total reimbursements equal to 0

3.6 Question 6: Join all employees with contributions and reimbursements

This question is one of the most important tasks in the lab because it requires combining the three datasets together: employees, contributions, and reimbursements.

To do this correctly, we first prepare the contributions data and the reimbursements data separately. For each employee, we calculate the total contribution amount and the total reimbursement amount. After obtaining these totals, we merge the results with the employees table. In this way, each employee appears with personal information, contribution information, and reimbursement information in a single final result.

This question is important because it represents a complete integration of the available data. It allows us to build one unified view of each employee by merging the tables instead of studying each file separately. As a result, we can compare contributions and reimbursements for the same employee in a clear and organized way.

```

2026-04-21 15:33:30,910 [main] INFO org
o process : 1
(E001,Walid,M,3495.36,167.6)
(E002,Rima,F,2353.08,)
(E003,Ryad,M,1962.79,109.63)
(E004,Rima,F,3181.63,354.1)
(E005,Karim,M,5035.84,)
(E006,Lara,F,3558.29,217.73)
(E007,Ryad,M,4054.86,)
(E008,Sara,F,6709.469999999999,392.91)
(E009,Hichem,M,6354.280000000001,)
(E010,Yara,F,1653.61,248.64)
(E011,Fares,M,5296.01,)
(E012,Houda,F,2765.84,219.43)
(E013,Anis,M,2566.02,)
(E014,Maya,F,6485.7,)
(E015,Henry,M,2135.35,)
(E016,Isla,F,2033.23,221.94)
(E017,Ryad,M,6807.58,)
(E018,Sana,F,851.31,)
(E019,Bob,M,7614.58,)
(E020,Yara,F,3168.96,72.6)
(E021,Omar,M,2076.95,83.08)

```

Figure 7: Result of Question 6: Join all employees with contributions and reimbursements

Execution analysis of Query 6 in YARN and MapReduce

For Query 6, Pig first analyzed the script and detected that the main features used were HASH_JOIN and GROUP_BY. This is expected because the query first computes totals for contributions and reimbursements, and then joins these results with the employees table.

The following line shows this clearly:

```

2026-04-21 15:57:40,601 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig
features used in the script: HASH_JOIN, GROUP_BY

```

After that, Pig compiled the query into a MapReduce execution plan. The next lines show that the plan contained four MapReduce jobs, both before and after optimization:

```

2026-04-21 15:57:40,642 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer -
MR plan size before optimization: 4

```

```
2026-04-21 15:57:40,642 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer -  
MR plan size after optimization: 4
```

This means that Query 6 was more complex than Query 1 and required four separate MapReduce jobs to complete all grouping and join operations.

Then Pig contacted YARN through the ResourceManager in order to request resources and submit the jobs to the cluster:

```
2026-04-21 15:57:40,659 [main] INFO
```

```
org.apache.hadoop.yarn.client.DefaultNoHARMFailoverProxyProvider - Connecting to  
ResourceManager at /192.168.20.153:8032
```

Pig then detected the reduce phase and estimated the number of reducers needed for the first aggregation jobs:

```
2026-04-21 15:57:40,662 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler -  
Reduce phase detected, estimating # of required reducers.
```

```
2026-04-21 15:57:40,664 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler -  
Setting Parallelism to 1
```

These lines show that Pig recognized that grouping operations needed reducers, and it selected one reducer for the job because the dataset size was relatively small.

After preparing the runtime environment, Pig announced that the first two jobs were ready for submission:

```
2026-04-21 15:57:41,275 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 2  
map-reduce job(s) waiting for submission.
```

These first two jobs correspond to the aggregation of reimbursements and contributions separately.

The first aggregation job was submitted to YARN for reimbursements:

```
2026-04-21 15:57:41,489 [JobControl] INFO org.apache.hadoop.mapreduce.JobSubmitter  
- Submitting tokens for job: job_1770582555454_1911
```

```
2026-04-21 15:57:41,518 [JobControl] INFO
org.apache.hadoop.yarn.client.api.impl.YarnClientImpl - Submitted application
application_1770582555454_1911
```

```
2026-04-21 15:57:41,781 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -
HadoopJobId: job_1770582555454_1911
```

```
2026-04-21 15:57:41,781 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -
Processing aliases g6r,r6,reb
```

This job grouped the reimbursements data by employee and produced the total reimbursement amount for each employee.

The second aggregation job was submitted to YARN for contributions:

```
2026-04-21 15:57:41,738 [JobControl] INFO org.apache.hadoop.mapreduce.JobSubmitter
- Submitting tokens for job: job_1770582555454_1912
```

```
2026-04-21 15:57:41,967 [JobControl] INFO
org.apache.hadoop.yarn.client.api.impl.YarnClientImpl - Submitted application
application_1770582555454_1912
```

```
2026-04-21 15:57:41,973 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -
HadoopJobId: job_1770582555454_1912
```

```
2026-04-21 15:57:41,973 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -
Processing aliases c6,con,g6c
```

This second job grouped the contributions data by employee and produced the total contribution amount for each employee.

The global progress of these two jobs was then shown by Pig:

```
2026-04-21 15:57:41,983 [main] INFO
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -
0% complete
```

2026-04-21 15:57:51,187 [main] INFO

org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 25% complete

2026-04-21 15:57:56,209 [main] INFO

org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 50% complete

After both aggregation jobs finished successfully, Pig prepared the third job, which joins the employees table with the aggregated contributions result:

2026-04-21 15:57:57,806 [main] INFO

org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 1 map-reduce job(s) waiting for submission.

2026-04-21 15:57:58,040 [JobControl] INFO org.apache.hadoop.mapreduce.JobSubmitter - Submitting tokens for job: job_1770582555454_1913

2026-04-21 15:57:58,271 [JobControl] INFO

org.apache.hadoop.yarn.client.api.impl.YarnClientImpl - Submitted application application_1770582555454_1913

2026-04-21 15:57:58,310 [main] INFO

org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1770582555454_1913

2026-04-21 15:57:58,310 [main] INFO

org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Processing aliases a6,emp,j6a

This job produced an intermediate relation containing employee information together with total contributions.

The progress of this third job was also reported:

2026-04-21 15:58:10,426 [main] INFO

org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 62% complete

```
2026-04-21 15:58:15,438 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -  
75% complete
```

After that, Pig prepared the fourth and final job. This job joined the previous intermediate result with the aggregated reimbursements result in order to build the final output:

```
2026-04-21 15:58:18,971 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 1  
map-reduce job(s) waiting for submission.
```

```
2026-04-21 15:58:19,214 [JobControl] INFO org.apache.hadoop.mapreduce.JobSubmitter  
- Submitting tokens for job: job_1770582555454_1914
```

```
2026-04-21 15:58:19,452 [JobControl] INFO
```

```
org.apache.hadoop.yarn.client.api.impl.YarnClientImpl - Submitted application  
application_1770582555454_1914
```

```
2026-04-21 15:58:19,475 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -  
HadoopJobId: job_1770582555454_1914
```

```
2026-04-21 15:58:19,475 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -  
Processing aliases j6b,q6
```

The progress of the final job was shown as follows:

```
2026-04-21 15:58:28,575 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -  
87% complete
```

```
2026-04-21 15:58:34,784 [main] INFO
```

```
org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher -  
100% complete
```

At the end, Pig reported the final script statistics and the summary of all four jobs:

2026-04-21 15:58:34,815 [main] INFO

org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:

Success!

job_1770582555454_1911 1 1 ... g6r,r6,reb GROUP_BY,COMBINER

job_1770582555454_1912 1 1 ... c6,con,g6c GROUP_BY,COMBINER

job_1770582555454_1913 2 1 ... a6,emp,j6a HASH_JOIN

job_1770582555454_1914 2 1 ... j6b,q6 HASH_JOIN

These lines show exactly how Query 6 was executed:

- Job 1911 grouped reimbursements by employee.
- Job 1912 grouped contributions by employee.
- Job 1913 joined employees with total contributions.
- Job 1914 joined the previous result with total reimbursements.

The final input and output statistics were then reported:

Successfully read 259 records (8658 bytes) from:

"/user/student207/shared/employee_contributions_small.csv"

Successfully read 100 records from: "/user/student207/shared/employees.csv"

Successfully read 60 records (2067 bytes) from:

"/user/student207/shared/employee_reimbursements.csv"

Successfully stored 100 records (3748 bytes) in:

"hdfs://192.168.20.153:9000/tmp/temp1826575340/tmp1965929394"

Total records written : 100

Finally, Pig displayed the execution dependency graph of the jobs:

job_1770582555454_1912 -> job_1770582555454_1913,

job_1770582555454_1913 -> job_1770582555454_1914,

job_1770582555454_1911 -> job_1770582555454_1914,

job_1770582555454_1914

This graph shows the exact order of execution. First, Pig computed the aggregated contributions and reimbursements in parallel. Then it joined employees with contributions, and finally it joined that intermediate result with reimbursements.

So, from a YARN and MapReduce point of view, Query 6 was executed as a multi-stage workflow composed of four MapReduce jobs. Pig decomposed the query into grouping and join phases, YARN allocated the cluster resources for each phase, Hadoop executed the jobs in the required order, and the final result produced one complete record for each employee containing employee information, total contributions, and total reimbursements.

3.7 Question 7: List female employees who have no reimbursements

This question focuses on female employees only and identifies those who did not receive any reimbursement. It combines filtering by gender with checking the absence of reimbursement data.

```
o process : 1
(E002,Rima)
(E014,Maya)
(E018,Sana)
(E022,Sana)
(E028,Hana)
(E034,Carol)
(E038,Carol)
(E044,Zineb)
(E046,Rania)
(E050,Alice)
(E052,Isla)
(E064,Soumia)
(E068,Lara)
(E074,Maya)
(E076,Amel)
(E078,Imane)
(E086,Dalia)
(E088,Dalia)
(E092,Mona)
(E096,Rania)
grunt> |
```

Figure 8: Result of Question 7: Female employees who have no reimbursements

3.8 Question 8: Show the top 3 contributors

The goal of this task is to find the three employees with the highest total contributions. This gives us a ranking of the strongest contributors in the dataset.

```
grunt> sh hdfs dfs -cat /user/student207/t  
E060,Carol,9504.88  
E026,Fatima,9015.67  
E083,Khaled,8579.59  
grunt> |
```

Figure 9: Result of Question 8: Top 3 contributors

3.9 Question 9: Count how many employees have zero reimbursement

This question is about calculating the total number of employees who did not receive any reimbursement. It gives a quantitative result instead of listing employees one by one.

```
s : 1  
2026-04-21 15:48:07,701 [main] INFO org.apache.p  
o process : 1  
(40)  
grunt> |
```

Figure 10: Result of Question 9: Count employees with zero reimbursement

3.10 Question 10: Count female employees with reimbursements equal to 0

This final task combines two conditions: the employee must be female, and the reimbursement value must be absent or equal to zero. The goal is to obtain the total number of female employees who did not benefit from reimbursements.

```
2026-04-21 15:52:47,873 [main] INFO o  
o process : 1  
(20)  
grunt> |
```

Figure 11: Result of Question 10: Count female employees with zero reimbursement

4 Conclusion

In this lab, we prepared the required data in HDFS and then used Apache Pig to answer the questions in Part 2. The work involved different big data operations such as filtering, counting, grouping, aggregation, and joining multiple datasets.

The most important parts of the lab were understanding how to extract specific employee groups, how to calculate totals from contribution and reimbursement data, and how to merge the datasets in order to obtain a complete analytical view. This practical work helped us better understand how Pig can be used to process structured data efficiently in a Hadoop environment.